

exo-gamma.bada – Bada量子エンジン (306行)

「関数における大域的部分積分多様体の機知 × Jones多項式の熱感知 – 地球と同位（同型）の地球外惑星と、電磁波を利用し得る生命体候補を探索する Bada 言語ソースコード

実装の対応表

要素	ソース内の実装
「の大域的部分積分多様体	gamma_fn (Lanczos近似) + gamma_check – 部分積分恒等式 $\Gamma(z+1)=z\Gamma(z)$ を実行時に検証 (=1.000000) し、esi3 の重み正規化に使用
Jones多項式の熱感知	jones_abs – 三葉結び目 $V(t)=-t^{-4}+t^{-3}+t^{-1}$ を $t=e^{i\theta}$ で評価。heat_theta が惑星の実測平衡温度→θ
地球と同位の惑星	esi3 (地球類似性指数) × 宇宙望遠鏡の実測カタログ8惑星 (Proxima b, TRAPPIST-1e, Kepler-442b ...)
生命体候補の発見	SETIスコア = ESI × 電波地平 (120光年) で「聞くべき候補」を順位付け – 検出は2026年時点でゼロと明記
Bada量子エンジン	qubit(2) → Hx2 → softmax振幅 → Measure (測定台帳へコミット)、CNOT の Bell対で零保存=盗聴なしを確認

実行方法: node bada_gui_ide/cli/bada-cli.js run bada_gui_ide/exo/exo-gamma.bada / ブラウザ内: GammaTwin (earth-twin.html)・PlanetCinema (planet-cinema.html) の「Bada量子エンジン」カード。 完走チャンネル: @@EX0-GAMMA-OK / @@GRAV-CHANNEL-OK / @@CIV-OK

```
1 # =====
2 # exo-gamma.bada – 電磁波を利用する地球型惑星の探索エンジン
3 # (GammaTwin / PlanetCinema の方程式を Bada
4 # 量子プログラミング言語で書き直したもの)
5 #
6 # ガンマ関数における大域的部分積分多様体の機知
7 #  $\Gamma(z) = \int t^{z-1} e^{-t} dt$  の部分積分から  $\Gamma(z+1) = z\Gamma(z)$ 
8 # – この関数等式(Lanczos 近似で実装)を ESI 重みの正規化と
9 # 自己検証に使う。sin / cos はテイラー級数、pow は exp・log で
10 # Bada 内に自作(組み込みは sqrt/log/exp/abs のみ)。
11 #
12 # Jones 多項式の熱感知
13 # 三葉結び目の  $V(t) = -t^{-4} + t^{-3} + t^{-1}$  を単位円  $t=e^{i\theta}$ 
14 # 上で評価し、θ を惑星の平衡温度から取る。|V| が熱感知の
15 # 強度(リポジトリの Jones 量子暗号と同じ結び目多項式族)。
16 #
17 # 電磁波を利用している地球型惑星(テクノシグネチャ)
18 # 人類自身が電波を漏らし始めて約120年。距離 d 光年の惑星は
19 # d <= 120 : 地球の電波が既に到達している(彼らが聞ける)
20 # d <= 60 : 往復可能 – 返信が今までに地球へ届き得る
21 # SETI スコア = ESI × 近接度。正直な注記: 地球以外の電磁波
22 # 利用は 2026 年時点で未確認。これは「聞くべき候補」の順位。
23 #
24 # 量子レジスタ (Bada の qubit / H / CNOT / Measure)
25 # 上位4候補を 2 量子ビットの重ね合わせに載せ、SETI スコアの
26 # softmax を振幅分布として測定台帳にコミット。Bell 対の
27 # 零保存 ( $|01\rangle, |10\rangle = 0$ ) を盗聴なしの証拠として確認する。
28 # =====
29
30 print("=====")
31 print("EX0-GAMMA BADA -- 「多様体 × Jones熱感知 × 量子レジスタ SETI探索」")
32 print("=====")
33
34 PI := 3.141592653589793
35
36 # ---- sin / cos: テイラー級数 (Bada には組み込みが無いので自作) ----
37 def sin_t(x) {
38   x := x % (2.0 * PI)
39   if (x > PI) { x := x - 2.0 * PI }
```

```

40     if (x < 0.0 - PI) { x := x + 2.0 * PI }
41     term := x
42     s := x
43     k := 1
44     while (k <= 11) {
45         term := 0.0 - term * x * x / ((2.0 * k) * (2.0 * k + 1.0))
46         s := s + term
47         k := k + 1
48     }
49     return s
50 }
51 def cos_t(x) { return sin_t(PI / 2.0 - x) }
52 def pow_f(b, e) {
53     if (b <= 0.0) { return 0.0 }
54     return exp(e * log(b))
55 }
56
57 # ---- 「関数: Lanczos 近似 (大域的部分積分の関数等式を検証) ----
58 def gamma_fn(z) {
59     g := [676.5203681218851, 0.0-1259.1392167224028, 771.32342877765313,
60         0.0-176.61502916214059, 12.507343278686905, 0.0-0.13857109526572012,
61         0.0000099843695780195716, 0.00000015056327351493116]
62     z := z - 1.0
63     x := 0.999999999999980993
64     i := 0
65     while (i < 8) {
66         x := x + g[i] / (z + i + 1.0)
67         i := i + 1
68     }
69     t := z + 7.5
70     return sqrt(2.0 * PI) * pow_f(t, z + 0.5) * exp(0.0 - t) * x
71 }
72 def gamma_check(w) { return gamma_fn(1.0 + w) / (w * gamma_fn(w)) }
73
74 print("")
75 print("[Γ] 部分積分の関数等式  $\Gamma(z+1)=z\Gamma(z)$  の検証 (=1 が正解):")
76 print("    w=0.57 :", f5(gamma_check(0.57)))
77 print("    w=0.70 :", f5(gamma_check(0.70)))
78 print("    w=5.58 :", f5(gamma_check(5.58)))
79 gh := gamma_fn(0.5)
80 print("     $\Gamma(0.5)^2 =$ ", f5(gh * gh), " ( $\pi =$ ", f5(PI), ")")
81
82 # ---- Jones 多項式 (三葉結び目) の熱感知 ----
83 def jones_abs(theta) {
84     re := 0.0 - cos_t(0.0 - 4.0 * theta) + cos_t(0.0 - 3.0 * theta) + cos_t(0.0 - theta)
85     im := 0.0 - sin_t(0.0 - 4.0 * theta) + sin_t(0.0 - 3.0 * theta) + sin_t(0.0 - theta)
86     return sqrt(re * re + im * im)
87 }
88 def heat_theta(teq) {
89     t := teq
90     if (t < 100.0) { t := 100.0 }
91     if (t > 1500.0) { t := 1500.0 }
92     return (t - 100.0) / 1400.0 * PI
93 }
94
95 # ---- ESI (地球類似性指数) - 「恒等式で重みを正規化 ----
96 def esi3(rade, insol, teq) {
97     s1 := 1.0 - abs(rade - 1.0) / (rade + 1.0)
98     s2 := 1.0 - abs(insol - 1.0) / (insol + 1.0)
99     s3 := 1.0 - abs(teq - 255.0) / (teq + 255.0)
100    w1 := 0.57 * gamma_check(0.57)      # ×1.000000 (恒等式)
101    w2 := 0.70 * gamma_check(0.70)
102    w3 := 5.58 * gamma_check(5.58)
103    return pow_f(s1, w1 / 3.0) * pow_f(s2, w2 / 3.0) * pow_f(s3, w3 / 3.0)
104 }

```

```

105
106 # ---- 電磁波（テクノシグネチャ）－ 地球の電波地平 ----
107 RADIO_YEARS := 120.0          # 人類の電波漏えい開始 ~1906 年から
108 def em_status(dist_ly) {
109     if (dist_ly <= RADIO_YEARS / 2.0) { return 2 } # 往復可能
110     if (dist_ly <= RADIO_YEARS)         { return 1 } # 地球の電波が到達済み
111     return 0
112 }
113 def em_label(s) {
114     if (s == 2) { return "往復可（返信が届き得る）" }
115     if (s == 1) { return "地球の電波が到達済み" }
116     return "電波地平の外"
117 }
118 def seti_score(e, dist_ly) { return e * (1.0 / (1.0 + dist_ly / 60.0)) }
119
120 # ---- 惑星カタログ（宇宙望遠鏡の実測値）-----
121 # [名前, R0, S0, Teq K, 距離 光年, 主星Teff K, 公転周期 日, 重力チャンネル]
122 # 重力チャンネル = 1: 惑星自身の重力で発見された（視線速度法 = 恒星のふらつき）
123 #                  = 0: 光度チャンネル（トランジット = 減光）で発見された
124 P0 := ["Proxima Centauri b", 1.07, 0.65, 234.0, 4.25, 3042.0, 11.19, 1]
125 P1 := ["Teegarden's Star b", 1.04, 1.15, 264.0, 12.50, 2904.0, 4.91, 1]
126 P2 := ["Ross 128 b", 1.11, 1.38, 280.0, 11.00, 3192.0, 9.87, 1]
127 P3 := ["Luyten b (GJ 273 b)", 1.51, 1.06, 259.0, 12.40, 3382.0, 18.60, 1]
128 P4 := ["GJ 1002 b", 1.03, 0.67, 231.0, 15.80, 3024.0, 10.30, 1]
129 P5 := ["TRAPPIST-1 e", 0.92, 0.65, 250.0, 40.66, 2566.0, 6.10, 0]
130 P6 := ["TOI-700 d", 1.07, 0.87, 269.0, 101.40, 3480.0, 37.40, 0]
131 P7 := ["Kepler-442 b", 1.34, 0.70, 233.0, 1206.00, 4402.0, 112.30, 0]
132 planets := [P0, P1, P2, P3, P4, P5, P6, P7]
133
134 # ---- 重力チャンネル：電磁波以外の伝達 -----
135 # 惑星は自らの重力場で恒星を振り回し、その「ふらつき」が我々に届く。
136 # 視線速度の半振幅（実式,  $\sin i = 1$ ):
137 #  $K = 0.08946 \text{ m/s} \times (M_p/M_\oplus) \times (M_\star/M_\odot)^{-2/3} \times (P/\text{1yr})^{-1/3}$ 
138 # 質量は半径から概算  $M_p \approx R^{3.58}$ （岩石惑星の質量-半径関係の近似）、
139 # 主星質量は  $M_\star \approx (T_{\text{eff}}/5772)^2$ （主系列の粗い近似）。いずれも「概算」。
140 def mass_est(rade) {
141     if (rade > 6.0) { return 100.0 + rade * 8.0 } # 巨大ガス惑星の粗い概算
142     return pow_f(rade, 3.58)
143 }
144 def star_mass(teff) { return pow_f(teff / 5772.0, 2.0) }
145 def rv_K(rade, teff, per_days) {
146     mp := mass_est(rade)
147     ms := star_mass(teff)
148     pyr := per_days / 365.25
149     return 0.08946 * mp * pow_f(ms, 0.0 - 2.0 / 3.0) * pow_f(pyr, 0.0 - 1.0 / 3.0)
150 }
151
152 # ---- 反重力場：実在する斥力的重力 = ダークエネルギー（宇宙定数  $\Lambda$ ）-----
153 # 加速膨張の実測 ( $H_0 \approx 70 \text{ km/s/Mpc}$ ,  $\Omega_\Lambda \approx 0.7$ )。軌道半径  $r$  での
154 # 反重力加速度  $a_\Lambda \approx \Omega_\Lambda \cdot H_0^2 \cdot r$  (de Sitter 斥力)
155 # 恒星の重力  $a_\star = G M_\star / r^2$ 
156 # の比を実計算し、惑星系スケールで反重力伝達が可能かを正直に判定する。
157 def antigrav_ratio(teff, insol) {
158     H0 := 0.00000000000000000000002268 # 70 km/s/Mpc [1/s]
159     G := 0.00000000000667 #  $\text{m}^3/(\text{kg s}^2)$ 
160     MSUN := 1989000000000000000000000000000.0
161     AUm := 149600000000.0
162     aAU := sqrt(pow_f(teff / 5772.0, 4.0) / insol)
163     r := aAU * AUm
164     aL := 0.7 * H0 * H0 * r
165     aS := G * star_mass(teff) * MSUN / (r * r)
166     return aL / aS
167 }
168
169 print("")

```

[illegible]

```

235 best1 := 0.0 - 1.0
236 best2 := 0.0 - 1.0
237 best3 := 0.0 - 1.0
238 name0 := ""
239 name1 := ""
240 name2 := ""
241 name3 := ""
242 while (i < n) {
243     p := planets[i]
244     e := esi3(p[1], p[2], p[3])
245     jv := jones_abs(heat_theta(p[3]))
246     st := em_status(p[4])
247     sc := seti_score(e, p[4])
248     print("    ", p[0], "| ESI", f5(e), "| |V|", f5(jv), "|", em_label(st), "| SETI", f5(sc))
249     if (sc > best0) {
250         best3 := best2
251         name3 := name2
252         best2 := best1
253         name2 := name1
254         best1 := best0
255         name1 := name0
256         best0 := sc
257         name0 := p[0]
258     } else { if (sc > best1) {
259         best3 := best2
260         name3 := name2
261         best2 := best1
262         name2 := name1
263         best1 := sc
264         name1 := p[0]
265     } else { if (sc > best2) {
266         best3 := best2
267         name3 := name2
268         best2 := sc
269         name2 := p[0]
270     } else { if (sc > best3) {
271         best3 := sc
272         name3 := p[0]
273     } } } }
274     i := i + 1
275 }
276
277 # ---- 量子レジスタ：上位4候補を重ね合わせて測定 ----
278 print("")
279 print("[量子] 上位4候補を 2 qubit の重ね合わせに載せる (Hx2):")
280 reg := qubit(2)
281 reg := H(reg, 0)
282 reg := H(reg, 1)
283 uni := Measure(reg)
284 print("    一様重ね合わせ |psi|^2 :", f5(uni[0]), f5(uni[1]), f5(uni[2]), f5(uni[3]))
285 amp := softmax([best0 * 8.0, best1 * 8.0, best2 * 8.0, best3 * 8.0])
286 amp := Measure(amp)
287 print("    SETI 重み付き振幅 (softmax*8, 測定台帳へコミット):")
288 print("    ", name0, ":", f5(amp[0]))
289 print("    ", name1, ":", f5(amp[1]))
290 print("    ", name2, ":", f5(amp[2]))
291 print("    ", name3, ":", f5(amp[3]))
292 print("    量子優先ターゲット =", name0)
293
294 # ---- Bell 対の零保存 = 盗聴なしの証拠 (QKD 同型) ----
295 bell := qubit(2)
296 bell := H(bell, 0)
297 bell := CNOT(bell, 0, 1)
298 bo := Measure(bell)
299 intact := 0

```

```
300 if (bo[1] == 0) { if (bo[2] == 0) { intact := 1 } }
301 print("")
302 print("[Bell] 零保存 |01>, |10> = 0 (盗聴なし=1):", intact, " 台帳 =", len(tuplespace), "facts")
303 print("")
304 print("正直な注記: 地球以外の電磁波利用は 2026 年時点で未確認。上記は")
305 print("宇宙望遠鏡の実測値から計算した「聞くべき候補」の優先順位である。")
306 print("@@EXO-GAMMA-OK")
```

Bada quantum programming language – github.com/masaaki-avnturle/Bada · [bada_gui_ide/exo/exo-gamma.bada](#) · generated 2026-09-18